Wireless Networks — Course Project

Rate-Limited Sender Increase in MsQuic CUBIC

Implementing and evaluating
`draft-ietf-ccwg-ratelimited-increase-11`

Abhiram Suresh

November 2026

Code. MsQuic branch `rate-limited-increase` (commits `dd0428a62`, `19538311e`, `7c6397cf3`), on top of upstream `464c873c0`. Patch files are in `patches/`; experiment scripts in `experiments/`; everything needed to rebuild this document and its figures is in `report/`.

Abstract

A *rate-limited* sender is one that is not using its whole congestion window: a video player sending one segment every few hundred milliseconds, an RPC client, a game, a heartbeat. TCP and QUIC grow the congestion window (*cwnd*) on every acknowledgement, so a rate-limited sender can keep growing *cwnd* for a long time even though the network never carried that much of its data. When the application later has a lot to send, it releases the whole inflated window at once, overflowing the bottleneck queue and hurting every other flow sharing it. The IETF draft *Increase of the Congestion Window when the Sender Is Rate-Limited* bounds *cwnd* by the largest flight the sender actually used since the last window reduction, *maxFS*. MsQuic's CUBIC already had a bound of this kind, but with two gaps: *maxFS* was never reset after a congestion event, and the bound was $2 \cdot maxFS$ in both slow start and congestion avoidance. We close both gaps (36 lines of code), add four unit tests, and evaluate the change in two ways: a deterministic, packet-by-packet scenario harness with an *ablation* that disables one change at a time, and a 10 Mbit/s, ≈41 ms dumbbell testbed with a competing flow. In the testbed, after 15 s of rate-limited sending the patch cuts *cwnd* at resumption from 324 to 182 KB, the data lost after resuming from 103 to 21 KB, and the worst competing round-trip time from 283 to 199 ms, while the 8 MB bulk transfer that follows takes the same time (6.84 s vs 6.85 s).

Contents

1 Introduction: why this matters **2**

2 Background **3**
2.1 Congestion window, flight size and slow start . . . 3
2.2 Why CUBIC is especially exposed . . . 3
2.3 The draft's rules . . . 3

3 What stock MsQuic does, and the two gaps **3**
3.1 A worked example of both gaps . . . 4

4 Implementation **4**
4.1 A design decision: *ssthresh* is left alone . . . 6
4.2 Instrumentation . . . 6

5 Per-change examples: the scenario harness and ablation **6**
5.1 Scenario A — resuming after a long rate-limited period (C1, C2, C3) . . . 6
5.2 Scenario B — why *ssthresh* is not lowered (design decision) . . . 7
5.3 Scenario C — a spurious loss is undone (C5) . . . 8
5.4 Scenario D — a heartbeat sender (C4 and the design decision) . . . 8
5.5 Scenario E — ECN (C1 and C2 on the ECN path) . . . 8

6 Network experiment **9**
6.1 Testbed . . . 9
6.2 Workload . . . 9
6.3 Results . . . 9

7 Unit tests **11**

8 Discussion and limitations **11**

9 Conclusion **12**

A Audit of the earlier plots in `experiments/` **13**

B Reproducing the results **14**

1 Introduction: why this matters

Most real Internet traffic is not a long bulk transfer. It comes in bursts separated by quiet periods, and between the bursts the sender has less to send than its congestion window would allow. Examples:

- **Adaptive video.** A player downloads a few seconds of video, then waits for the buffer to drain, then downloads again.
- **Request/response protocols** (HTTP/3, RPC). The sender is limited by how fast requests arrive, not by the network.
- **Real-time and interactive traffic** (games, telemetry, heartbeats). The sender emits a few small packets per round trip.

The congestion window exists to answer one question: *how much data can this path safely carry right now?* The only evidence the sender has is the data it actually sent and got acknowledged. If the sender keeps only 29 KB in flight but lets *cwnd* grow to 400 KB, the extra 371 KB is a guess nobody checked. The problem shows up when the application suddenly has a lot to send (the next video chunk, a large response): the sender is allowed to release the whole window in one round trip. That burst lands in the bottleneck queue, the queue overflows, packets of *other* flows are dropped and their latency jumps.

Figure 1 is the headline result of this project, measured on a real network stack (Section 6). With stock MsQuic, a competing ping that normally sees 43 ms sees up to 366 ms in the second after our sender resumes; with the patch, it sees at most 194 ms in the same run, and the bulk transfer that caused the burst is not slower.

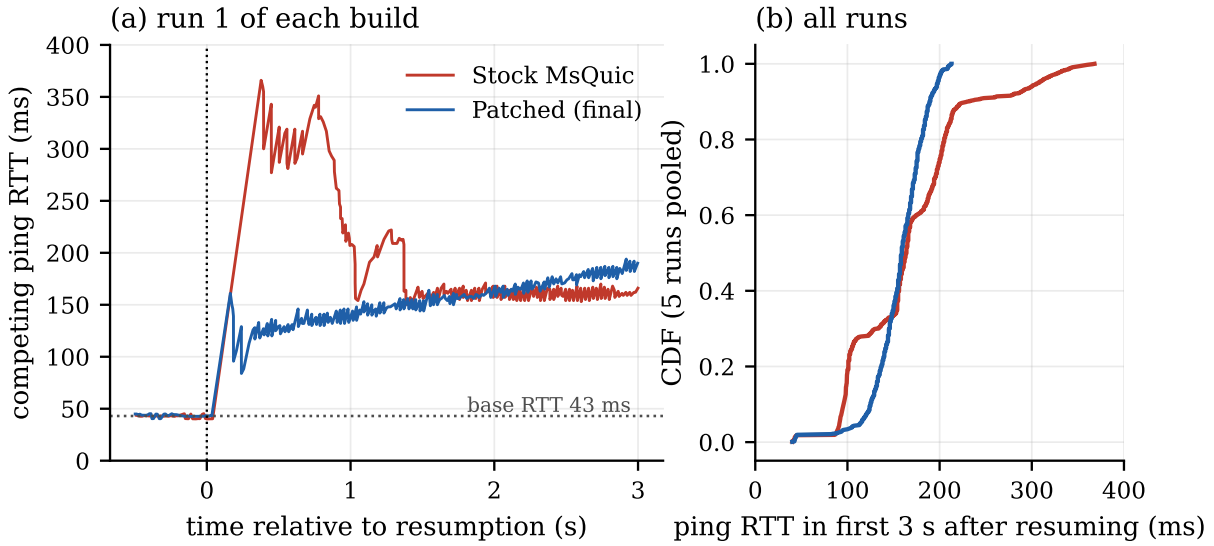


Figure 1: Latency seen by a competing flow when the rate-limited QUIC sender resumes bulk sending at $t = 0$. (a) One run of each build. (b) Cumulative distribution of every ping RTT in the first 3 s after resuming, pooled over five runs per build. Details in Section 6.

Contributions.

1. An analysis of where MsQuic's CUBIC deviates from the draft (two gaps, Section 3).
2. A patch that closes both gaps, made of five small changes C1–C5 (Section 4), plus four new and two updated unit tests.
3. A deterministic scenario harness and an ablation study that shows, for *each* change separately, the exact situation in which it matters and what goes wrong without it (Section 5).

4. A testbed experiment showing the end-to-end effect on a competing flow (Section 6).
5. An audit of the earlier exploratory plots in `experiments/` (Appendix A).

2 Background

2.1 Congestion window, flight size and slow start

The sender may have at most *cwnd* bytes unacknowledged. *FlightSize* is the number of bytes actually in flight. A sender is *cwnd-limited* when *FlightSize* $\approx$ *cwnd* and *rate-limited* (also called application-limited) when *FlightSize* $<$ *cwnd* because the application has nothing more to send. In *slow start* (*cwnd* $<$ *ssthresh*) the window doubles every round trip; in *congestion avoidance* (*cwnd* $\geq$ *ssthresh*) it grows slowly. On a loss or ECN mark the window is reduced (by CUBIC's $\beta = 0.7$) and *ssthresh* is set to the new window.

2.2 Why CUBIC is especially exposed

CUBIC (RFC 9438) computes the window as a function of *time since the last congestion event*, $W(t) = C\,(t - K)^3 + W_{\max}$, not as a function of the number of ACKs. As long as ACKs keep arriving, the window follows this curve. A rate-limited sender still receives ACKs (for its few packets), so the window climbs the cubic curve exactly as if the path had been fully used. The longer the rate-limited period, the larger the unvalidated window. In our testbed the stock window grew from 170 KB to 430 KB during 11 s in which only 29 KB was ever in flight (Figure 8).

2.3 The draft's rules

`draft-ietf-ccwg-ratelimited-increase-11` updates RFC 5681, RFC 9002 and others with a simple rule. It introduces

$$maxFS = \text{the largest } \textit{FlightSize} \text{ since the last time } \textit{cwnd} \text{ was reduced,}$$

and requires that, while rate-limited, the window does not grow past what *maxFS* justifies:

Table 1: The rules of the draft that this project implements.

Rule	Requirement
R1 Reset	When *cwnd* is reduced for any reason, *maxFS* is reset to 0 and re-measured from the following *FlightSize*.
R2 Slow start	$cwnd \leq 2 \cdot maxFS$ (allows one doubling over what was used).
R3 Congestion avoidance	$cwnd \leq maxFS + 1 \cdot SMSS$ (allows the normal one-packet-per-RTT probe over what was used).

These rules are the successor to Congestion Window Validation (RFC 7661), which instead *decays cwnd* after idle/rate-limited periods. The draft takes the simpler approach of never letting the window run ahead of the evidence in the first place.

3 What stock MsQuic does, and the two gaps

MsQuic's CUBIC (`src/core/cubic.c`) already tracks *maxFS*, in a field called `BytesInFlightMax`. It is updated on every send (`OnDataSent`) and is used at the end of `OnDataAcknowledged` to cap the window:

Stock `cubic.c`, end of `CubicCongestionControlOnDataAcknowledged`

```
// Using 2 * BytesInFlightMax for the limit allows for exponential growth
// in the window when not otherwise limited.
if (Cubic->CongestionWindow > 2 * Cubic->BytesInFlightMax) {
    Cubic->CongestionWindow = 2 * Cubic->BytesInFlightMax;
}
```

The field is set to $cwnd/2$ at initialisation and on reset, and otherwise only ever grows. Compared with Table 1:

Gap 1: *maxFS* is never reset (violates R1). After a loss the window is cut to 70 %, but *maxFS* keeps the pre-loss peak. The cap $2 \cdot maxFS$ is then computed from a flight size that the network just showed it *cannot* carry.

Gap 2: the congestion-avoidance cap is $2 \cdot maxFS$ (violates R3). In congestion avoidance the window may be twice the largest flight, instead of one packet more than it.

3.1 A worked example of both gaps

Take the situation of Scenario A (Section 5.1). Units are packets of 1232 B.

1. The sender is cwnd-limited and reaches $cwnd = 40$, with 40 packets in flight. So $maxFS = 40$.
2. 12 of the 40 are lost. CUBIC cuts the window to $cwnd = ssthresh = 0.7 \cdot 40 = 28$.
3. The application now sends only 4 packets per RTT for 10 seconds (200 RTTs of 50 ms).

Table 2: Window limits in the worked example after the loss. With stock MsQuic, CUBIC's time-driven growth fills all the headroom it is allowed.

	maxFS after loss	cap in CA	*cwnd* after 10 s at 4 pkt/RTT
Stock (Gap 1 + Gap 2)	40 (stale)	$2 \cdot 40 = 80$	**76.5**
Draft rules	$0 \to 28$ (reset, re-measured)	$28 + 1 = 29$	**29**

With stock MsQuic the sender ends up with a 76-packet window, *bigger than the 40 packets that just caused a loss*, although it never had more than 4 packets in flight after the loss. When the application resumes, all 76 packets go out in one round trip.

4 Implementation

The patch touches only `cubic.c` (+36 lines of code, +29 lines of comments) and adds one field to `cubic.h`. It consists of five changes. Figure 2 shows where each one sits in the CUBIC callbacks.

C1 — reset *maxFS* on every congestion event (closes Gap 1)

`CubicCongestionControlOnCongestionEvent` is the single place where CUBIC reduces the window for loss, ECN and persistent congestion, so one line there implements R1 for all of them. The old value is saved first so that a spurious loss can be undone (C5).

```
    Cubic->PrevBytesInFlightMax = Cubic->BytesInFlightMax;
 ...
// "upon a reduction of cwnd (for any reason), maxFS MUST be reset to zero"
Cubic->BytesInFlightMax = 0;
```

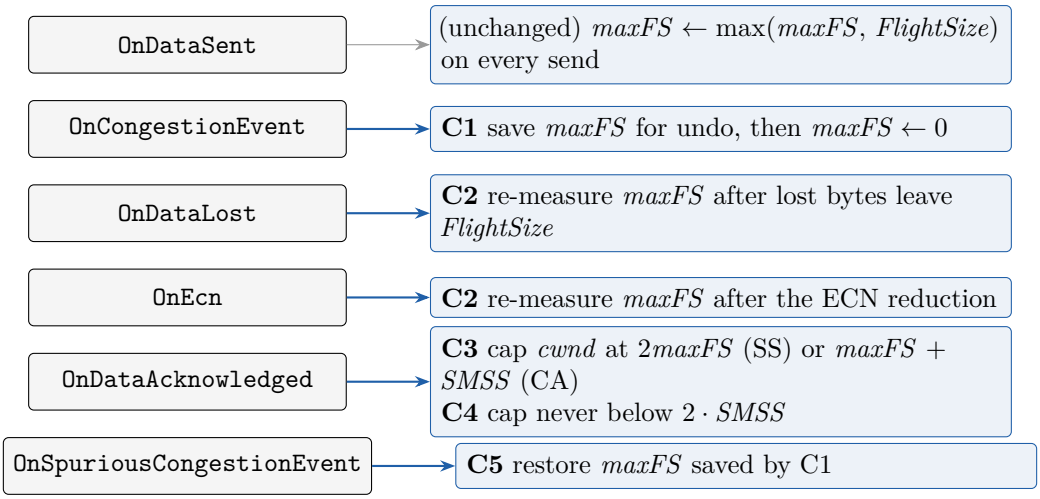


Figure 2: The CUBIC callbacks and the change made in each (`OnDataLost` and `OnEcn` both call `OnCongestionEvent`). Blue boxes are new behaviour; the grey box is existing behaviour the patch relies on.

C2 — re-measure *maxFS* right after the reduction

After a reset to 0, the draft says *maxFS* is measured again from the next *FlightSize*. If we waited for the next `OnDataSent`, any ACK arriving before that would see $maxFS = 0$. So both callers of the congestion event measure it immediately, at the point where *FlightSize* is already correct: in `OnDataLost` *after* the lost bytes have been subtracted (otherwise lost packets would count as "carried"), and in `OnEcn` after the reduction.

```
static void
CubicCongestionControlUpdateMaxFlightSize(_In_ QUIC_CONGESTION_CONTROL_CUBIC* Cubic)
{
    if (Cubic->BytesInFlightMax < Cubic->BytesInFlight) {
        Cubic->BytesInFlightMax = Cubic->BytesInFlight;
    }
}
```

C3 — a phase-dependent cap (closes Gap 2), and C4 — a floor

```
    if (Cubic->CongestionWindow > 2 * Cubic->BytesInFlightMax) {
        Cubic->CongestionWindow = 2 * Cubic->BytesInFlightMax;
    }
    const uint32_t Smss = QuicPathGetDatagramPayloadSize(&Connection->Paths[0]);
    const BOOLEAN InSlowStart = Cubic->CongestionWindow < Cubic->SlowStartThreshold;
    uint32_t CwndLimit;
    if (InSlowStart) {
        CwndLimit = 2 * Cubic->BytesInFlightMax;          // R2
    } else {
        CwndLimit = Cubic->BytesInFlightMax + Smss;       // R3
    }
    const uint32_t MinWindow = Smss * QUIC_PERSISTENT_CONGESTION_WINDOW_PACKETS;
    if (CwndLimit < MinWindow) {                          // C4
        CwndLimit = MinWindow;
    }
    if (Cubic->CongestionWindow > CwndLimit) {
        Cubic->CongestionWindow = CwndLimit;
    }
```

C4 is not in the draft's text but is required by QUIC: RFC 9002 says the window must never

go below the minimum window of 2 packets (`QUIC_PERSISTENT_CONGESTION_WINDOW_PACKETS`). Without it, a sender whose flight is a single small packet would get a window smaller than one full packet (Scenario D).

C5 — restore *maxFS* when a loss turns out to be spurious

MsQuic undoes a congestion event when the "lost" packets are acknowledged after all. It already restored *cwnd*, *ssthresh* and the CUBIC state from `Prev*` copies. Because C1 now changes *maxFS* on the congestion event, it has to be restored too, otherwise the first ACK after the undo would cap the restored window with the post-loss *maxFS* (Scenario C).

```
uint32_t PrevBytesInFlightMax;                    // cubic.h
Cubic->BytesInFlightMax = Cubic->PrevBytesInFlightMax;   // OnSpuriousCongestionEvent
```

4.1 A design decision: *ssthresh* is left alone

When C3 lowers the window below *ssthresh*, the sender is technically back in slow start, and on the next ACK the slow-start cap $2 \cdot maxFS$ applies, so *cwnd* can rise again until it reaches *ssthresh*. The window therefore oscillates, but always within $\min(ssthresh, 2 \cdot maxFS)$.

An earlier version of the patch also lowered *ssthresh* to the capped window to stop the oscillation. The ablation (Scenario B and D) showed this is harmful: *ssthresh* is the sender's memory of the last *real* congestion, and lowering it without any congestion signal makes the sender leave slow start early when the application later becomes cwnd-limited. In Scenario D, the sender reached 23 packets after 15 RTTs with *ssthresh* untouched and only 9 with it lowered (Figure 6). The draft also does not ask for it. We therefore reverted it (commit `7c6397cf3`). In the testbed runs this branch never fired (*ssthresh* only changed at loss events), so the network results hold for both versions.

4.2 Instrumentation

For the experiments, a CSV logger (`CUBIC_CWND_LOG`) records *cwnd*, *maxFS*, *FlightSize* and *ssthresh* at every send, ACK and loss. It is compiled only with `-DQUIC_CUBIC_CWND_LOG` and has no effect on normal builds. The sample client (`src/tools/sample/sample.c`) gained options to produce a controlled rate-limited workload (`-prime`, `-burst`, `-pause`, `-cycles`, `-final`) and prints the connection statistics at the end.

5 Per-change examples: the scenario harness and ablation

Network experiments show the overall effect but cannot tell which change is responsible, and they are noisy. To show precisely *where* each change matters, we drive CUBIC directly through its callbacks inside MsQuic's own unit-test harness. Every scenario is a deterministic script of sends, ACKs and losses (one round = one 50 ms RTT, $SMSS = 1232$ B, HyStart off), and is run against seven builds of `cubic.c`:

Table 4 summarises the outcome; the subsections explain each scenario. Reading the table column by column shows that every change has a scenario in which removing it, and only it, changes the result.

5.1 Scenario A — resuming after a long rate-limited period (C1, C2, C3)

Setup. The worked example of Section 3.1: slow start to 40 packets, 12 lost, 200 rounds at 4 packets per round, then the application becomes cwnd-limited and sends a full window per round.

Table 3: Builds used in the ablation. Each "no_" build is the final patch with exactly one change switched off by a compile-time macro.

Build	Meaning
stock	upstream `cubic.c`
full	the final patch (C1–C5)
no_reset	without C1/C2 (*maxFS* never reset)
no_cacap	without the CA part of C3 (cap stays $2 \cdot$ *maxFS*)
no_floor	without C4
no_restore	without C5
ssthresh_lower	the rejected design of Section 4.1

Table 4: Ablation summary (units: packets of 1232 B unless stated). Bold marks the build in each column that differs from full because the change under test is missing.

	A: burst on resume	C: *cwnd* on 1st ACK after undo	D: *cwnd* in heartbeat phase	D: *cwnd* after 15 RTTs of bulk	E: *maxFS* after ECN
stock	76	29.5	25 861 B	25	40
full	29	29.5	2 464 B	23	20
no_reset	**41**	29.5	**25 861 B**	25	**40**
no_cacap	**56**	29.5	2 464 B	23	20
no_floor	29	29.5	**600 B**	22	20
no_restore	29	**22.0**	2 464 B	23	20
ssthresh_lower	29	29.5	2 464 B	**9**	20

What happens. Figure 3(a) shows the window during the rate-limited period. Stock CUBIC follows its cubic curve up to 76.5 packets. Removing either half of the fix still leaves a large window:

- *no_reset* (Gap 1 only fixed in CA): *maxFS* stays 40, the CA cap is $40 + 1 = 41$.
- *no_cacap* (Gap 1 fixed, Gap 2 not): *maxFS* is correctly 28, but the cap is $2 \cdot 28 = 56$.
- *full*: *maxFS* is 28 and the cap is $28 + 1 = 29$, one packet above *ssthresh*, exactly what R3 allows.

Figure 3(b) shows the consequence: the number of packets released in the first round after resuming. The patched sender sends 29, the stock sender 76, i.e. 2.6 times more, onto a path that dropped packets at 40.

5.2 Scenario B — why *ssthresh* is not lowered (design decision)

Setup. Like A, but 20 of the 40 packets are lost, so the re-measured $maxFS = 20$ and the CA cap $20 + 1 = 21$ is *below* $ssthresh = 28$. 40 rate-limited rounds, then 15 cwnd-limited rounds.

What happens. Figure 4. With the final patch, the cap drops *cwnd* to 21, the sender is then in slow start and the $2 \cdot maxFS$ cap lets *cwnd* rise back to *ssthresh*; the window moves between 21 and 28 and never exceeds *ssthresh*. When the application resumes, the sender is already in slow start and ramps to 31 packets in 15 rounds. With *ssthresh* lowered to 21, the window is flat during the rate-limited period but the sender then grows by only one packet every two rounds, reaching 27. The stock sender is at 34–36 throughout, above the 28 that *ssthresh* says is safe.

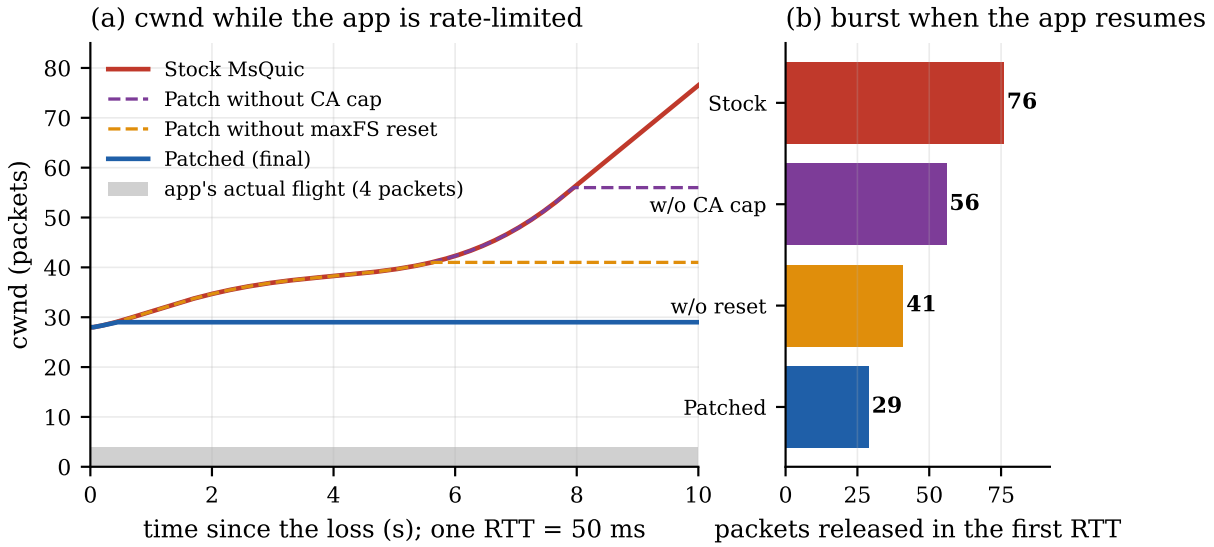


Figure 3: Scenario A. (a) *cwnd* during 200 rounds in which only 4 packets per round are sent. (b) Burst size in the first round after the application resumes. Both fixes are needed; each alone leaves a burst well above the 40 packets at which the loss occurred.

5.3 Scenario C — a spurious loss is undone (C5)

Setup. In congestion avoidance with *cwnd* ≈ 29 and a full window in flight, 30 % of the flight is declared lost; then MsQuic learns the loss was spurious and calls `OnSpuriousCongestionEvent`; then the rest of the flight is acknowledged.

What happens. Figure 5. The undo restores *cwnd* to 29.3 in every build. But C1 had set *maxFS* to the post-loss flight of 21. Without C5, the first ACK applies the CA cap $21 + 1 = 22$ and the undo is silently cancelled: the sender loses 25 % of its window for a loss that never happened. With C5, *maxFS* is restored to 36 and the window continues at 29.5, as in stock.

5.4 Scenario D — a heartbeat sender (C4 and the design decision)

Setup. After a loss at 40 packets, the application sends one 300-byte heartbeat per round for 25 rounds; one heartbeat is lost. Then the application starts a bulk transfer.

What happens. Figure 6(a): during the heartbeat phase the stock window stays at 25 861 B (21 full packets) although the flight is 300 B: when the bulk transfer starts it sends 20 packets at once. With the draft rules the cap is $maxFS + SMSS = 300 + 1232$, which C4 raises to the QUIC minimum window of $2 \cdot 1232 = 2464$ B. Without C4 the window falls to 600 B, *smaller than one full-size packet*, which RFC 9002 forbids and which costs the sender an extra round trip when it resumes.

Figure 6(b) shows the ramp-up when the bulk transfer starts. The full patch slow-starts 2, 4, 8, 16 and reaches *ssthresh* in four rounds, then grows in congestion avoidance. The rejected *ssthresh*-lowering variant has *ssthresh* at 2 packets and grows by one packet every two rounds: 9 packets after 15 rounds instead of 23.

5.5 Scenario E — ECN (C1 and C2 on the ECN path)

Setup. A 40-packet window is in flight; 20 packets have been acknowledged when an ECN Congestion Experienced mark arrives. 100 rate-limited rounds at 4 packets follow.

What happens. Table 5. ECN reduces the window from 60 to 42 packets in every build. The patched sender then re-measures *maxFS* from the 20 packets still in flight; the stock sender keeps 40. During the rate-limited period the stock window keeps climbing the cubic curve to 56.6 packets; the patched window never exceeds *ssthresh* (42) and ends at 37.

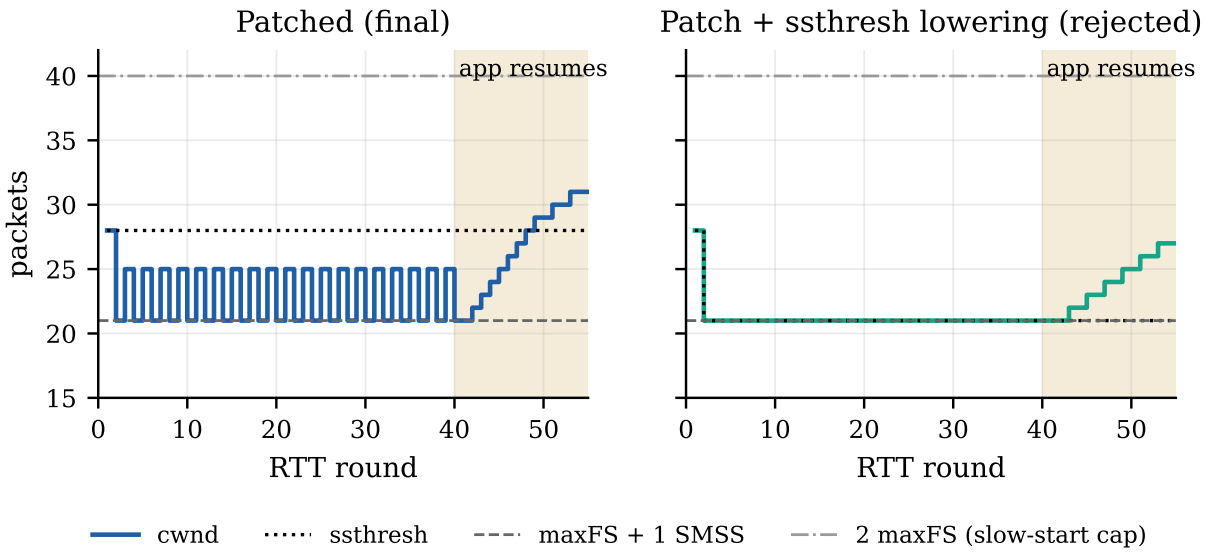


Figure 4: Scenario B. Left: the final patch. Right: the rejected variant that lowers *ssthresh*. The oscillation of the final patch stays below *ssthresh* and costs nothing; lowering *ssthresh* slows the later ramp-up.

Table 5: Scenario E (packets).

	cwnd before ECN	*cwnd* after ECN	*maxFS* after ECN	*cwnd* after 100 rate-limited rounds
stock	60	42	40	56.6 (still growing)
full	60	42	20	37 (bounded by *ssthresh*)

6 Network experiment

6.1 Testbed

Linux network namespaces form a dumbbell (Figure 7). The bottleneck is the r1–r2 link, shaped by `netem` in both directions: 10 Mbit/s, 20 ms delay in each direction, a 100-packet queue. The measured base RTT is 41 ms for QUIC and 43 ms for ping; the bandwidth-delay product is about 52 KB. A second client pings a second server through the same bottleneck every 10 ms to measure what our sender does to other traffic.

6.2 Workload

The sample client runs one connection with three phases:

1. **Prime:** 3 MB bulk, so the window grows and loses packets as in normal operation.
2. **Rate-limited:** 1500 cycles of 6000 B followed by a 10 ms pause (about 4.8 Mbit/s offered, ≈15 s). The flight stays at 29 KB in both builds.
3. **Resume:** 8 MB bulk, sent as fast as *cwnd* allows.

Each build is run five times, alternating stock and patched. Time 0 in all figures is the moment the 8 MB transfer starts.

6.3 Results

Figure 8 shows one complete run of each build. In the stock run, *maxFS* (dashed) stays at the 830 KB peak of the prime phase for the whole experiment (Gap 1). During the rate-limited

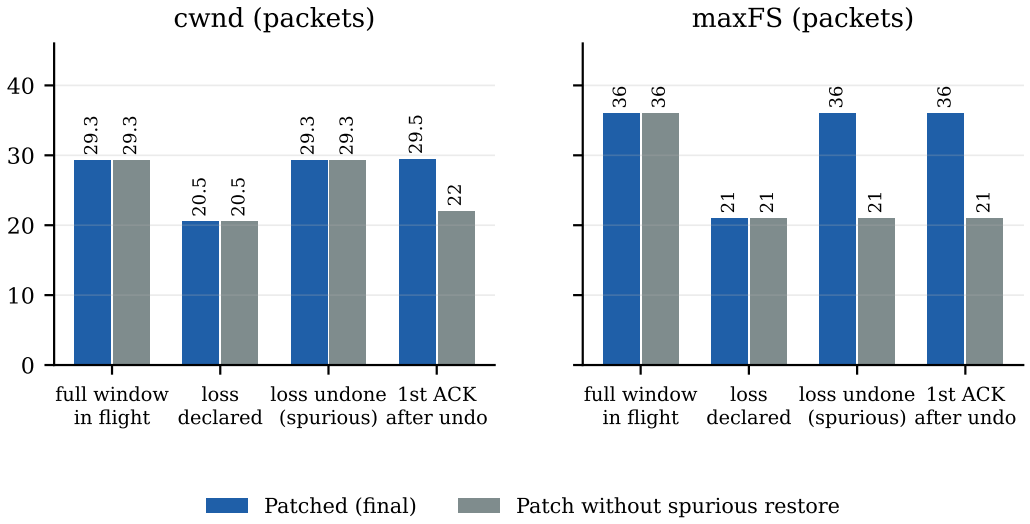


Figure 5: Scenario C. *cwnd* and *maxFS* at each step of a spurious loss and its undo. Without C5 the restored window is capped on the next ACK.

period, with 29 KB in flight, *cwnd* climbs from 170 to 430 KB, because the cap $2 \cdot maxFS = 1.7$ MB is never reached. At $t = 0$ the sender releases that window, the queue overflows and *cwnd* is cut three times in a row. In the patched run, *maxFS* is re-measured after each loss and *cwnd* is held near 170 KB during the rate-limited period.

Table 6: Testbed results, five runs per build: mean and [min–max].

Metric	Stock	Patched	Change
Flight during rate-limited phase	29 KB	29 KB	—
maxFS at resumption	849 [832–874] KB	180 [168–199] KB	−79 %
cwnd at resumption	324 [244–426] KB	182 [169–200] KB	−44 %
cwnd / flight at resumption	11.2×	6.3×	
Peak flight in first 0.5 s	334 [253–436] KB	195 [183–213] KB	−42 %
Data lost after resuming	103 [21–224] KB	21 [7–26] KB	−79 %
Loss events after resuming	22 [9–41]	10 [3–12]	−56 %
Max competing ping RTT (first 3 s)	283 [218–368] ms	199 [186–213] ms	−30 %
95th-pct. ping RTT (first 3 s)	261 ms	190 ms	−27 %
Time to send the final 8 MB	6.85 s	6.84 s	none

Table 6 and Figure 9 summarise all ten runs. The patch reduces the window at resumption, the burst, the losses it causes and the latency it inflicts on the competing flow, and it does so *without* costing throughput: the 8 MB transfer takes the same time, because the patched sender ramps up from a validated window within a few round trips, while the stock sender spends the same time recovering from the losses its burst caused.

The stock results have a wide range for a clear reason: CUBIC's window grows with time since the last loss. In stock runs 1–2 the last loss was about 14 s before resumption and *cwnd* reached 426 KB; in runs 3–5 it was about 11 s and *cwnd* reached 244–276 KB. The longer an application is rate-limited, the worse stock CUBIC behaves. The patched results are tight because they depend only on what was actually sent.

What did not change. The total number of bottleneck drops over the whole run (460 vs 423) and the number of lost ping packets (27 vs 29) are similar, because the prime phase and the

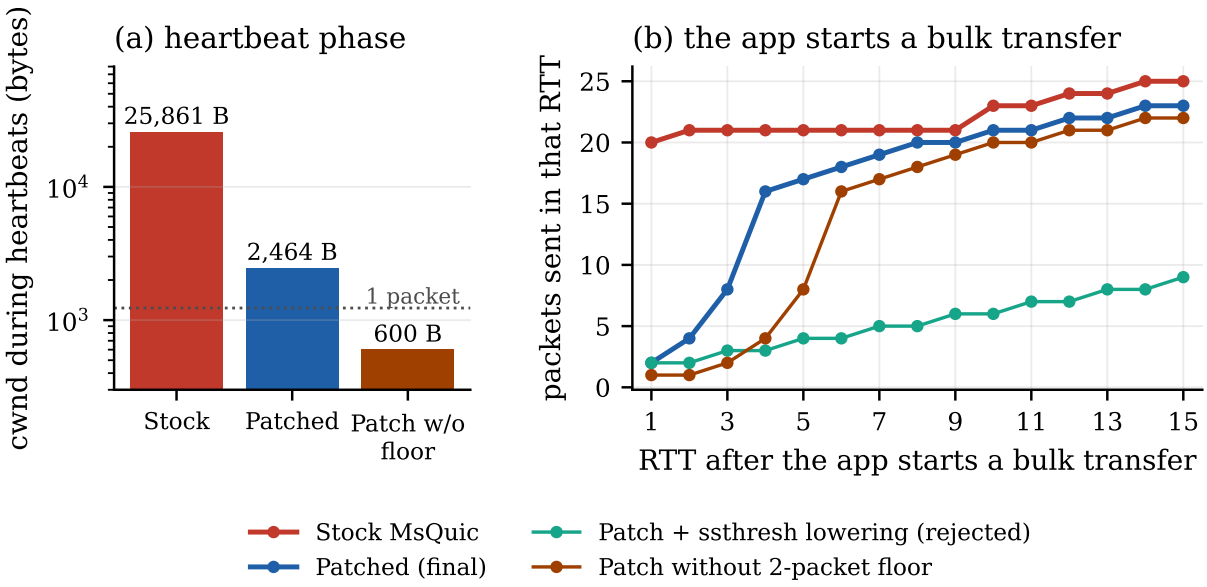


Figure 6: Scenario D. (a) Window while one 300 B heartbeat is sent per round (log scale). (b) Packets sent per round once the bulk transfer starts.

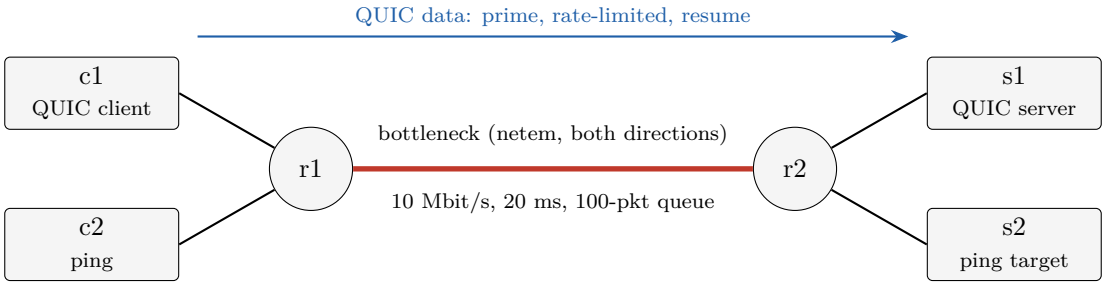


Figure 7: Dumbbell topology. The QUIC client uploads to s1; c2's pings share the bottleneck queue.

8 MB transfer dominate them. The patch addresses the burst at resumption, not steady-state bulk behaviour, which it leaves unchanged.

7 Unit tests

All 56 tests of MsQuic's `CubicTest` suite pass with the patch. Table 7 lists the tests that were added or changed. Each new test fails on stock MsQuic.

8 Discussion and limitations

- **Scope.** Only CUBIC is changed. MsQuic's BBR has its own application-limited handling and is not affected.
- **Oscillation.** Without lowering *ssthresh*, a rate-limited sender's window can move between $maxFS + SMSS$ and $\min(ssthresh, 2 \cdot maxFS)$ (Scenario B). This is bounded by validated values and was the better trade-off (Section 4.1).
- **SMSS.** The cap uses the datagram payload size of path 0. MsQuic's CUBIC uses path 0 for all its window arithmetic, so this is consistent, but it is not multipath-aware.
- **Testbed.** One emulated path, one competing flow, five runs per build. `netem`'s queue limit counts socket buffers; with UDP GSO one buffer can hold several packets, so the real

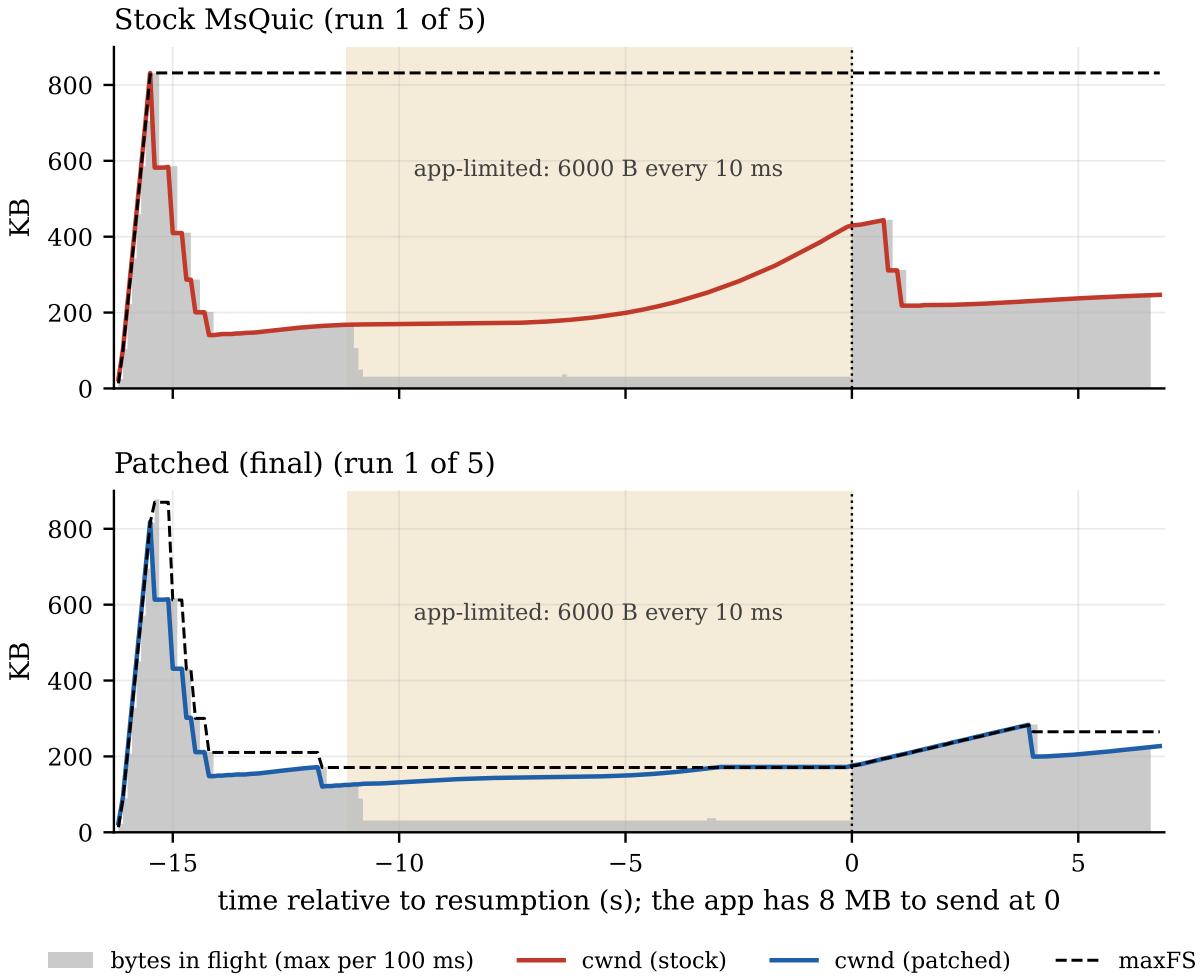


Figure 8: *cwnd*, *maxFS* and bytes in flight for run 1 of each build. The shaded band is the rate-limited phase (only its last 11 s are shown); the app resumes bulk sending at $t = 0$.

queue was probably longer than 100 packets. The absolute delay and loss numbers depend on this detail; the comparison between builds is fair because both ran on the identical setup.

- **Scenario harness.** The scenarios use idealised whole-round ACKs. They are meant to isolate the logic of each change, not to model a network; the testbed covers that.

9 Conclusion

A rate-limited sender should not be allowed to claim capacity it never used. Stock MsQuic CUBIC already tried to enforce this, but its limit was based on a stale maximum and was twice as loose as the draft allows in congestion avoidance. With five small changes (36 lines of code) we implement the rules of `draft-ietf-ccwg-ratelimited-increase-11`. The ablation shows that each change is needed and exactly where it matters: C1–C3 stop the resumption burst (76 → 29 packets), C4 keeps the window legal for tiny flights, C5 keeps spurious-loss undo working, and leaving *ssthresh* alone keeps ramp-up fast. On a real stack and an emulated bottleneck, the patch cuts post-resumption losses by 79 % and the worst latency seen by a competing flow by 30 %, at no cost in transfer time.

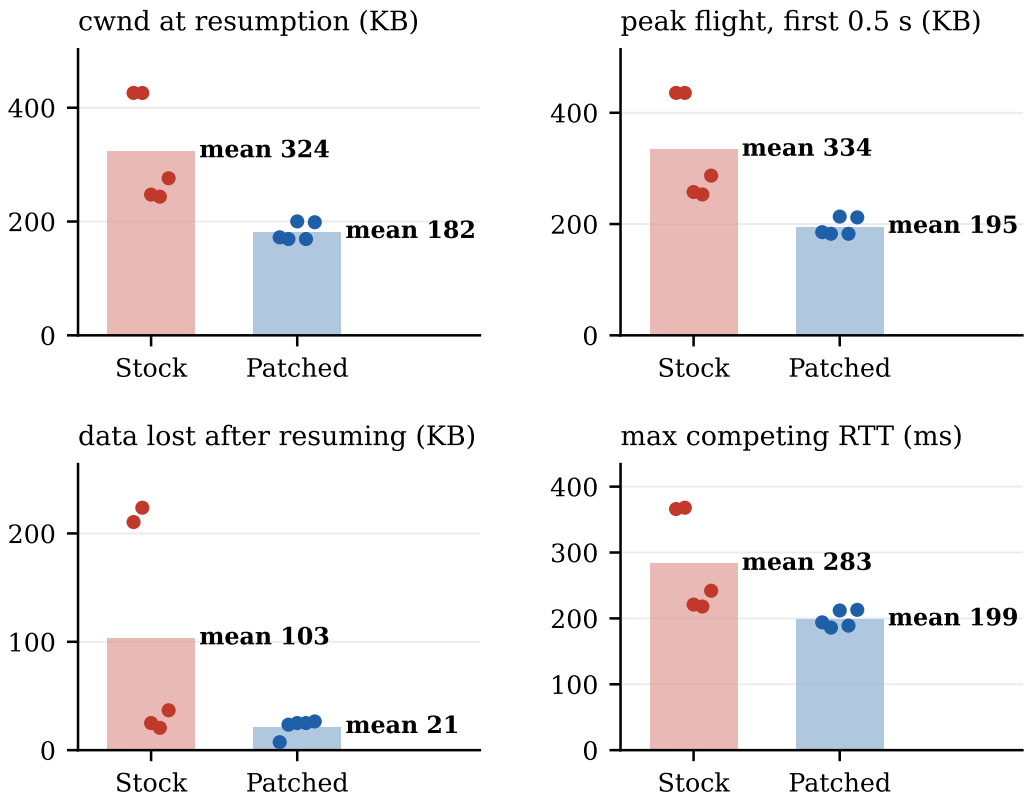


Figure 9: Per-run results. Bars are means, dots are the individual runs.

References

[1] G. Fairhurst, T. Jones, et al., *Increase of the Congestion Window when the Sender Is Rate-Limited*, IETF Internet-Draft `draft-ietf-ccwg-ratelimited-increase-11`.

[2] J. Iyengar, I. Swett, *QUIC Loss Detection and Congestion Control*, RFC 9002, 2021.

[3] L. Xu, S. Ha, I. Rhee, V. Goel, L. Eggert, *CUBIC for Fast and Long-Distance Networks*, RFC 9438, 2023.

[4] G. Fairhurst, A. Sathiaseelan, R. Secchi, *Updating TCP to Support Rate-Limited Traffic*, RFC 7661, 2015.

[5] M. Allman, V. Paxson, E. Blanton, *TCP Congestion Control*, RFC 5681, 2009.

[6] Microsoft, *MsQuic*, `https://github.com/microsoft/msquic`.

A Audit of the earlier plots in `experiments/`

Before this report, a number of exploratory plots had been made in `experiments/`. We checked each against the raw data and the code. None of them is used in this report; every figure here was regenerated by `report/make_figures.py` from the raw traces.

Table 7: New and updated unit tests in `src/core/unittest/CubicTest.cpp`.

Test	Change	What it checks
`RateLimited_MaxFsResetOnLoss`	C1, C2	After losing 2400 B out of 12 320 B in flight, *maxFS* equals the remaining 9920 B, not the old peak.
`RateLimited_MaxFsResetOnEcn`	C1, C2	After an ECN mark, *maxFS* equals the bytes still in flight.
`RateLimited_SpuriousCongestion-RestoresMaxFs`	C5	A spurious-loss undo restores *maxFS* to its pre-loss value.
`RateLimited_CongestionAvoidance-CapIsMaxFsPlusSmss`	C3	After 30 ACKs in congestion avoidance with a constant flight, $cwnd = maxFS + 1232$.
`SlowStart_WindowOverflowAfter-PersistentCongestion` (updated)	C3	Previously asserted the old $2 \cdot maxFS$ limit; now asserts $cwnd = maxFS + SMSS$ (8392 B).
`HyStart_ConservativeSlowStart-Rounds_TransitionToDone` (updated)	C3	Asserts $cwnd \le ssthresh$ and $cwnd \le 2 \cdot maxFS$ in slow start.

Table 8: Verdict on each earlier plot in `experiments/`.

Plot(s)	Verdict	Reason
`gap1_evidence_clean(_new)`, `gap1_harm_evidence`	correct	Show the stale *maxFS* correctly, but not its consequence; superseded by Figure 8.
`resumption_bomb*`, `bomb_attempt`	invalid	Client and server logs interleaved in one series, producing vertical spikes; titles claim a burst the plotted data does not show.
`before_cap_comparison*`, `gap1/gap2_after`	invalid	Caps computed from the plot title instead of the data, and from an earlier "+1 byte" version of the cap; `gap1_after` is from a superseded build.
`appendix_*`, `worked_example`, `rls_problem`	incomplete	Slow-start only (miss the CA rule) and mix bytes and packets on one axis; one has a 175 s axis artefact.

Conventions in the new figures. One colour per build throughout (stock red, patched blue); shared axes where panels are compared; units on every axis; the scenario figures in packets of 1232 B, the testbed figures in KB; every network figure states the run it shows or that it pools all five; all figures are vector PDFs.

B Reproducing the results

```
# apply the patches to upstream MsQuic 464c873c0
cd msquic && git am ../patches/*.patch
# unit tests
cmake --build build-test --target msquiccoretest
build-test/bin/Debug/msquiccoretest --gtest_filter='CubicTest.*'
# ablation (rebuilds cubic.c seven times, restores sources on exit)
report/ablation/run_ablation.sh
# testbed (needs root for network namespaces)
sudo experiments/run_rls_experiments.sh
python3 experiments/analyze_rls.py
# figures and this document
python3 report/make_figures.py
cd report && tectonic report.tex
```